

Crabe & Serpent : un mariage de raison

2026-05-01

1. Mise en bouche

Le constat

La stack Python qui va vite, aujourd'hui, est **écrite en Rust**.

- 🐼 Polars — DataFrame
- 🖌️ Ruff — linter
- 📦 uv — package manager
- ✅ Pydantic v2 — validation
- 😊 Tokenizers — NLP
- ⚡ orjson — JSON

Et ce n'est qu'un début...



Rust en bref

- **Performances** : sans GC, proche du métal
- **Sûreté mémoire** : ownership, borrow checker
- **Système de types** : enums, traits, lifetimes
- **Fearless concurrency** : data race = erreur de compilation
- **Bas niveau, mais ergonomique**



Python en bref

- **Souple, expressif, rapide à écrire**
- **Itération rapide** : prototypage, exploration, REPL
- **Accessibilité** : scolaire, scientifique, débutant
- **Écosystème colossal** : data, ML, web, scripting
- **Lent en pur Python** → libs natives pour la perf

NumPy, PyTorch, TensorFlow... déjà en C/C++. Et maintenant en Rust.

Quand Rust rencontre Python

Rust apporte	Python apporte
Perf et sûreté	Souplesse
Concurrence	Écosystème
Système de types	Accessibilité
Bas niveau	Prototypage

Le meilleur des deux mondes ?

2. Les fondations

ABI C, lingua franca

ABI $\neq$ API

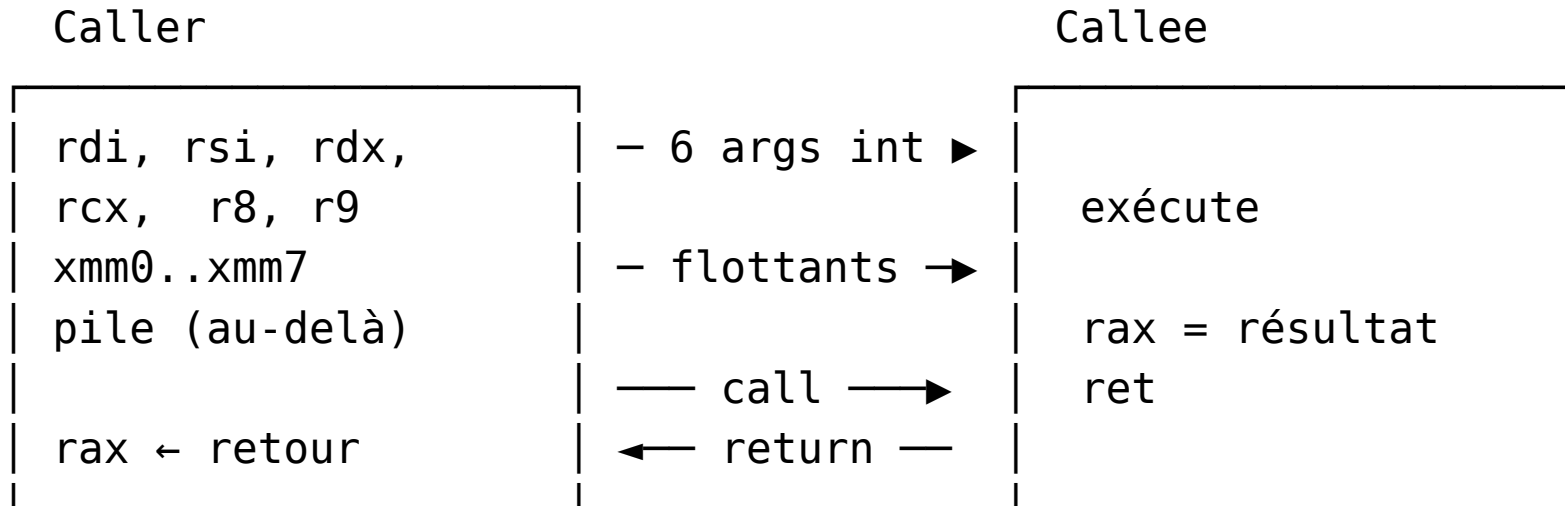
- Application **B**inary **I**nterface
- Convention de bas niveau : registres, pile, layout mémoire

Presque tous les langages **savent appeler C**.

→ Si Rust expose une fonction `extern "C"`, Python (ou n'importe quel langage) peut l'appeler.

Schéma d'un appel C

System V AMD64 (Linux/macOS) :



Frictions :

- Name mangling (extern "C")
- Layout (repr(C))

- Strings : `\0` vs `(ptr, len)`

Python : appeler du natif

```
import ctypes
lib = ctypes.CDLL("./libmath.so")
lib.add.argtypes = [ctypes.c_int, ctypes.c_int]
lib.add.restype = ctypes.c_int
print(lib.add(2, 3)) # 5
```

Approche	Pour	Contre
ctypes	Pas de compilation	Lent, peu sûr
cffi	Plus propre	Dépendance
C-API CPython	Perf max, intégration native	C, tout à la main
PyO3	Ergonomie + perf C-API	Compile Rust

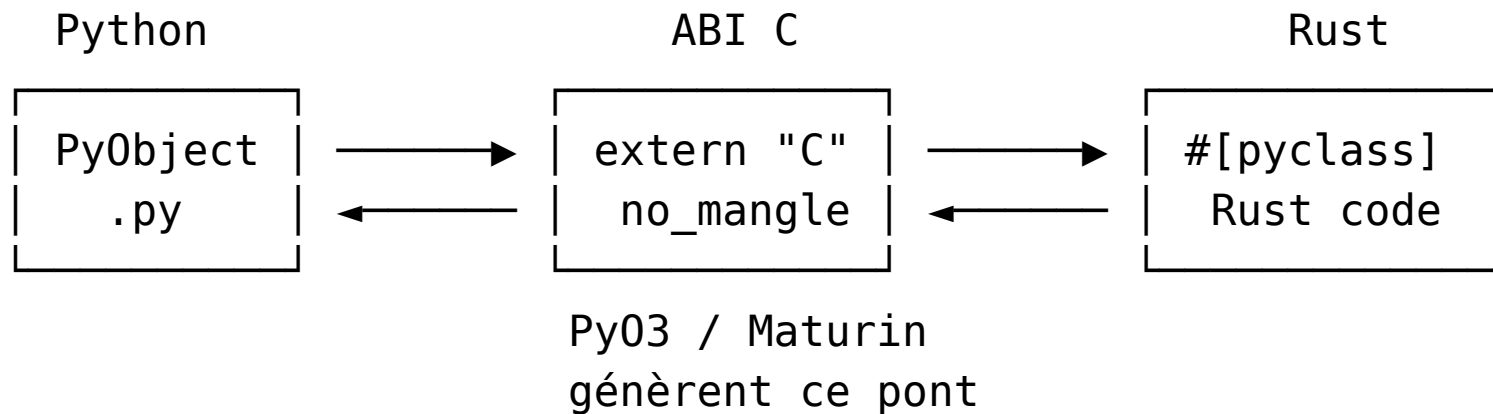
Rust : exposer du C

```
#[unsafe(no_mangle)]  
pub extern "C" fn add(a: i32, b: i32) -> i32 {  
    a + b  
}
```

```
#[repr(C)]  
pub struct Point {  
    x: f64,  
    y: f64,  
}
```

```
$ cargo build --release  
→ target/release/libmacrate.{so|dylib|dll}
```

Le pont



→ Vous écrivez du Rust idiomatique. → PyO3 + Maturin font tout le sale boulot.

3. L'écosystème



Le Rust gagne du terrain comme **moteur natif** :

-  Polars — DataFrame
-  Ruff — linter
-  uv — package manager
-  Pydantic v2 — validation
-  Tokenizers — NLP
-  orjson — JSON
-  DataFusion — SQL
-  Granian, Robyn — serveurs HTTP



L'inverse existe aussi :

- **Scripting / plugins** : embarquer un interpréteur Python pour la config, l'extension
- **ML/IA** : piloter PyTorch / scikit-learn depuis un service Rust
- **Migration progressive** : remplacer brique par brique sans tout réécrire
- **Outillage** : exposer du tooling Python (linters, formatters) à un binaire Rust

→ Moins courant, mais bien supporté par PyO3.

4. Du code, du vrai

La stack

PyO3 + Maturin

- **PyO3** : bindings Rust $\leftrightarrow$ Python (les macros)
- **Maturin** : build & packaging des wheels

```
$ maturin develop          # dev local
```

```
$ maturin build --release  # produit une wheel
```

Setup minimal

```
# Cargo.toml
```

```
[lib]
```

```
crate-type = ["cdylib"]
```

```
[dependencies]
```

```
pyo3 = { version = "0.28", features = ["abi3-py38"] }
```

```
# pyproject.toml
```

```
[build-system]
```

```
requires = ["maturin>=1.0"]
```

```
build-backend = "maturin"
```

→ C'est tout.

Exemple 1 — Markdown via comrak

```
use pyo3::prelude::*;
```

```
#[pyfunction]
```

```
fn markdown_to_html(input: &str) -> String {  
    comrak::markdown_to_html(input, &comrak::Options::default())  
}
```

```
#[pymodule]
```

```
fn rusty_md(m: &Bound<'_, PyModule>) -> PyResult<()> {  
    m.add_function(wrap_pyfunction!(markdown_to_html, m)?)?;  
    Ok(())  
}
```

```
from rusty_md import markdown_to_html  
print(markdown_to_html("# Hello **world**"))  
# <h1>Hello <strong>world</strong></h1>
```

Exemple 2 — Stats sur logs

```
#[pyfunction]
fn stats(durations_ms: Vec<f64>) -> (f64, f64, f64, f64) {
    let n = durations_ms.len() as f64;
    let sum: f64 = durations_ms.iter().sum();
    let avg = sum / n;
    let min = durations_ms.iter().copied()
        .fold(f64::INFINITY, f64::min);
    let max = durations_ms.iter().copied()
        .fold(f64::NEG_INFINITY, f64::max);
    let var = durations_ms.iter()
        .map(|x| (x - avg).powi(2))
        .sum::<f64>() / n;
    (avg, min, max, var.sqrt())
}
```

Bench Python vs Rust vs PyO3

```
import statistics, timeit
from log_analyze import stats as rust_stats

def py_stats(xs):
    return (statistics.mean(xs), min(xs),
            max(xs), statistics.stdev(xs))

xs = [...] # 1M de mesures
print("py: ", timeit.timeit(lambda: py_stats(xs), number=10))
print("rust:", timeit.timeit(lambda: rust_stats(xs), number=10))
```

À mesurer :

- Vec<f64> (copie) vs &PyList (zero-copy)
- Lire le log en Rust vs en Python
- Coût FFI s'amortit sur le volume

Exemple 3 — Python dans Rust

```
use pyo3::prelude::*;

fn main() -> PyResult<()> {
    Python::with_gil(|py| {
        let math = py.import("math")?;
        let pi: f64 = math.getattr("pi")?.extract()?;
        println!("π = {pi}");

        py.run(c"print('Hello from embedded Python')",
              None, None)?;
        Ok(())
    })
}

pyo3 = { version = "0.28", features = ["auto-initialize"] }
```

Ce qu'on peut faire

- ✓ **Modules** : `#[pymodule]`
- ✓ **Fonctions** : `#[pyfunction]`
- ✓ **Classes** : `#[pyclass]` + `#[pymethods]`
- ✓ **Exceptions custom** : `create_exception!`
- ✓ **Sous-modules**, conversions auto
- ✓ **NumPy** : `crate rust-numpy`
- ✓ **Async** : `pyo3-async-runtimes`
- ✓ **Type stubs** (`.pyi`) pour l'IDE / `mypy`

Ce qu'on ne peut pas (facilement)

// ❌ Génériques exposés tels quels

#[pyclass]

struct Container<T> { /* ... */ } // ne compile pas

// ✅ Solution : monomorphiser

#[pyclass] struct IntContainer { /* ... */ }

#[pyclass] struct StrContainer { /* ... */ }

// ❌ Iterator Rust → itérable Python (pas auto)

impl Iterator for MyType { /* ... */ }

// ✅ Implémenter __iter__ et __next__

#[pymethods]

impl MyType {

fn __iter__(slf: PyRef<'_, Self>) -> PyRef<'_, Self> { slf }

```
fn __next__(&mut self) -> Option<i32> { /* ... */ }  
}
```

5. REX : Toboggan-py

Architecture toboggan-py

```
toboggan-core/    ← logique métier pure (no_std capable)
toboggan-client/  ← client HTTP/WebSocket Rust
toboggan-py/      ← bindings Py03 (cdylib, abi3-py38)
└─ src/
    ├── lib.rs      #[pymodule]
    ├── toboggan.rs  #[pyclass] Toboggan
    ├── talk.rs, slides.rs, state.rs, ...
toboggan_py.pyi    ← type stubs (IDE, mypy)
```

→ **Le même cœur métier**, exposé en CLI, Web, Mobile, Python.

Le #[pyclass]

```
#[pyclass]
pub struct Toboggan {
    config: TobogganConfig,
    rt: Runtime,                // Tokio embarqué
    _ws: WebSocketClient,
    tx: UnboundedSender<Command>, // canal vers la tâche WS
    talk: Arc<RwLock<TalkResponse>>, // état partagé
    slides: Arc<RwLock<SlidesResponse>>,
    state: Arc<RwLock<TState>>,
}
```

Le **pattern clé** :

Python sync, Rust async, Runtime Tokio embarqué.

Les #[pymethods]

```
#[pymethods]
impl Toboggan {
    #[new]
    #[pyo3(signature = (host = "localhost", port = 8080))]
    pub fn __new__(host: &str, port: u16) -> PyResult<Self> {
        // ... setup tokio runtime, websocket, channels
    }

    #[getter]
    pub fn state(&self) -> PyResult<State> {
        let state = Arc::clone(&self.state);
        let s = self.rt.block_on(async {
            state.read().await.clone()
        });
        Ok(State(s))
    }
}
```

```
}
```

```
pub fn next(&self) { self.send(Command::NextSlide); }
```

```
pub fn previous(&self) { self.send(Command::PreviousSlide); }
```

```
}
```

Côté Python

```
from toboggan_py import Toboggan
```

```
tbg = Toboggan("localhost", 8080)
```

```
print(f"talk: {tbg.talk}")
```

```
print(f"state: {tbg.state}")
```

```
tbg.next()           # avance d'une slide
```

```
tbg.previous()       # recule
```

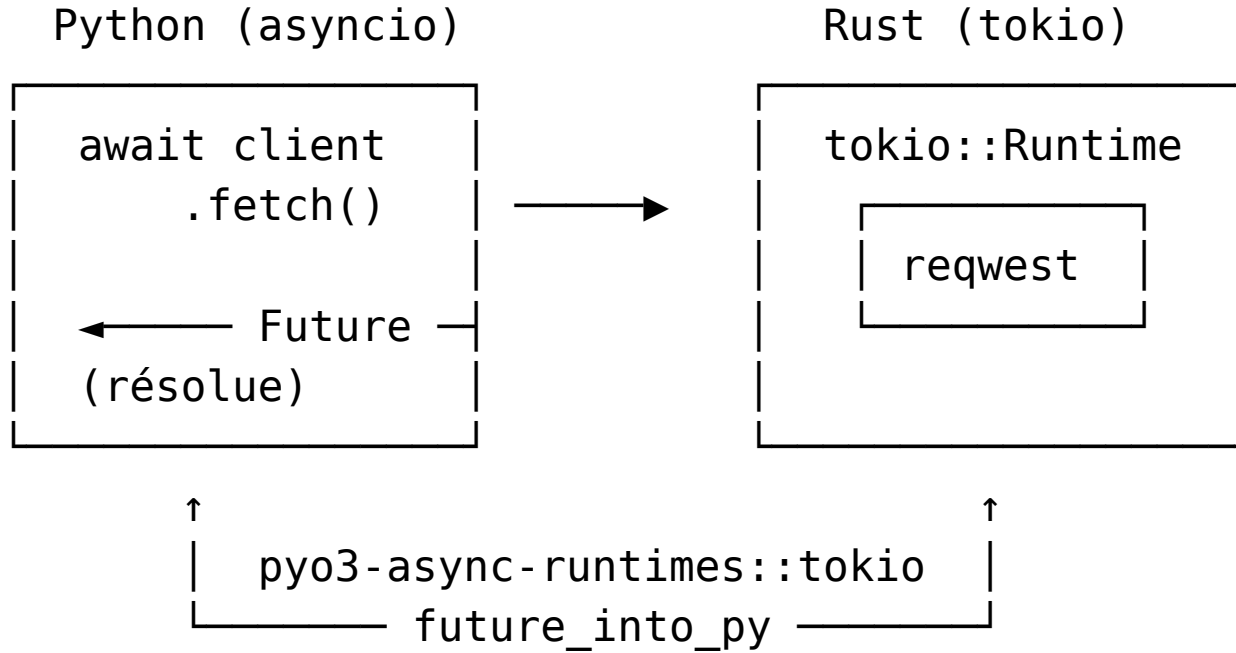
```
tbg.goto(5)          # slide précise
```

```
tbg.blink()           # effet visuel
```

→ **API Pythonique, moteur Rust, WebSocket temps réel.** → Cette présentation est pilotée par ce code.

6. Les écueils

Asynchronisme



Deux mondes async **indépendants** → il faut un pont.

Deux patterns

// Pattern 1 : Runtime interne, API Python sync

#[pymethods]

```
impl Client {  
    fn fetch(&self, url: &str) -> PyResult<String> {  
        self.rt.block_on(async {  
            request::get(url).await?.text().await  
        }).map_err(|e| PyRuntimeError::new_err(e.to_string()))  
    }  
}
```

// Pattern 2 : pyo3-async-runtimes, API Python async

#[pyfunction]

```
fn fetch(py: Python<'_>, url: String) -> PyResult<Bound<'_, PyAny>> {  
    pyo3_async_runtimes::tokio::future_into_py(py, async move {  
        request::get(&url).await  
        .map_err(|e| PyRuntimeError::new_err(e.to_string()))?  
    })  
}
```

```
        .text().await
        .map_err(|e| PyErr::new_err(e.to_string()))
    })
}
```

OpenTelemetry

Python (otel-python)

```
trace_id: ABC
span_id: 001
|
| — Py03 call —>
|
| ← return —
```

Rust (opentelemetry-rust)

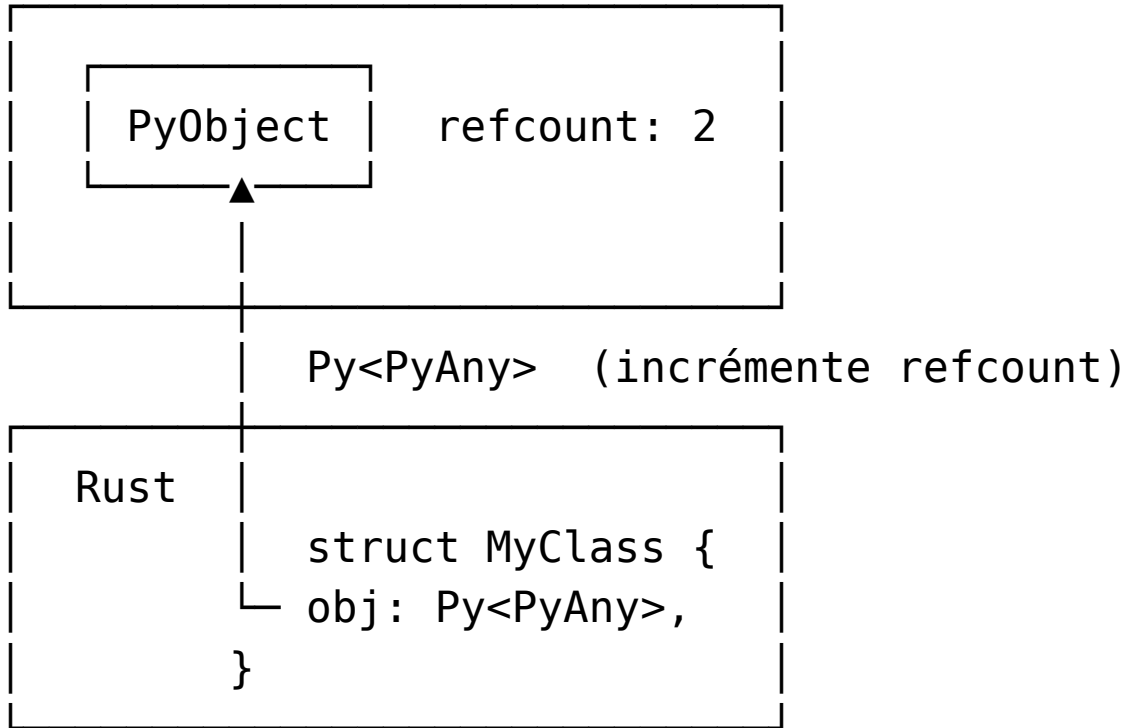
```
trace_id: ???
span_id: ???
|
|
| X contexte perdu
```

Solution : injecter / extraire **W3C TraceContext** aux frontières.

Python : `propagate.inject(carrier)`
→ `{"traceparent": "00-ABC-001-01"}`
Rust : `Extractor` reconstruit le Context

Ownership : Python ou Rust ?

Python heap (gc, refcount)



- `Py<T>` = smart pointer Rust qui maintient une refcount Python

- GIL nécessaire pour **lire/écrire** un PyObject
-  Pas de cycle Rust $\leftrightarrow$ Python $\rightarrow$ fuite garantie

Le GIL et la PEP 703

Le Global Interpreter Lock :

- 1 thread Python à la fois exécute du bytecode
- Empêche le vrai parallélisme CPU-bound en pur Python
- Workaround : multiprocessing, ou du natif qui le relâche

PEP 703 — Free-threaded Python (3.13+)

- Le GIL devient **optionnel**
- Compile flag `--disable-gil`
- Vrai parallélisme CPU-bound

→ Les libs natives (PyO3 inclus) doivent s'adapter.

Packaging multi-plateforme

Wheels nécessaires :

- **manylinux** (glibc), **musllinux** (Alpine)
- **macOS** universal2 (arm64 + x86_64)
- **Windows** (x86_64, arm64)

```
# .github/workflows/release.yml
- uses: Py03/maturin-action@v1
  with:
    target: ${{ matrix.target }}
    args: --release --out dist
    manylinux: auto
- run: maturin upload --skip-existing dist/*
```

→ maturin-action orchestre le cross-compile + upload PyPI.

Tips de terrain

-  **Réduire la surface FFI** — moins de franchissements = moins d'overhead
-  **Batcher** plutôt que appels unitaires
-  **Runtime Tokio partagé** — `OnceLock<Runtime>`
-  **Libérer le GIL** sur le CPU-bound — `py.allow_threads(...)`
-  **Tester des deux côtés** — `pytest` ET `cargo test`
-  **Profiler avec `py-spy`** — voit les frames natives
-  **Type stubs `.pyi`** — IDE et `mypy` heureux

7. Conclusion

4 options stratégiques

1. **RIIR** — réécrire (partiellement) une lib Python en Rust pour la perf *ex : pydantic-core, polars*
2. **API polyglotte** — un cœur Rust, exposé partout (Python, Go, Java, ...) *ex : OpenDAL, Kreuzberg*
3. **Proto Python → Rust** — itérer vite, optimiser quand le besoin émerge
4. 🤔 **A-t-on encore besoin de Python ?** *Rust : polars, axum, candle, evcxr*

Pour aller plus loin

Outils

- PyO3, Maturin
- pyo3-async-runtimes, rust-numpy
- py-spy, cbindgen

Lectures

- Making Python 100x faster with Rust — Ohad Ravid
- PyO3 User Guide
- The Rustonomicon — FFI
- PEP 703 — GIL optional
- Astral blog — Ruff, uv

Code de la présentation

- github.com/ilaborie/toboggan

